Guia de Estado Actual
GhostMerc Frontier Territory

Cesar Josue Zapata Chilla

Marzo 2026

Contents

1 Que tienes ahora **3**
1.1 Pieza principal . . . 3
1.2 Stack activo . . . 3
1.3 Que ya no es el camino principal . . . 3

2 Arquitectura actual **3**
2.1 Separacion de responsabilidades . . . 3
2.2 Archivos importantes . . . 3

3 Que artefactos concretos tienes **4**
3.1 Demo maestra larga . . . 4
3.2 Resultados principales de esa demo . . . 4
3.3 Mejor episodio curado . . . 4

4 Que puedes ejecutar para ver ahora mismo **5**
4.1 Importante . . . 5
4.2 Para ver la demo maestra directamente . . . 5
4.3 Para ver el episodio exacto . . . 5
4.4 Para verlo con presentacion mas clara . . . 5
4.5 Para ver el corte editorial . . . 5
4.6 Para exportar video . . . 5
4.7 Para ver una politica viva . . . 5

5 Que puedes ejecutar para entrenar **6**
5.1 Baseline MLP . . . 6
5.2 Evaluacion del baseline . . . 6
5.3 GNN / backend relacional . . . 6
5.4 Master demo largo . . . 6

6 Que puedes ejecutar para la ruta RLHF **6**
6.1 Pipeline completo rapido . . . 6
6.2 Pipeline paso a paso . . . 6
6.3 Estado actual de la ruta de preferencias . . . 7

7 Que quedo validado tecnicamente **7**
7.1 CLI 7
7.2 Tests 7
7.3 Validaciones funcionales extra 7

8 Como interpretar lo que ves **8**
8.1 Proxy reward 8
8.2 True reward 8
8.3 Proxy-true gap 8
8.4 Armed neutral false positive rate 8
8.5 Containment tick count 8
8.6 Phase transition 8

9 Resumen rapido para explicarlo facil **8**
9.1 En una frase 8
9.2 En 30 segundos 8
9.3 Tres numeros que conviene memorizar 8

10 Que hacer despues **9**

11 Cierre **9**

1 Que tienes ahora

Ahora tienes un proyecto consolidado dentro de `PR_LR_H` con **Frontier Territory** como camino canonico. Ya no es una mezcla informal de demos: hay un entorno headless principal, entrenamiento PPO, evaluacion con reward oculto, renderer separado y una ruta inicial para GNN y preferencias.

1.1 Pieza principal

La pieza central del proyecto es:

- `chromahack/envs/ghostmerc_frontier_env.py`

Ese entorno ya soporta dos modos de observacion:

- **flat**: baseline MLP actual
- **dict**: ruta estructurada para el backend relacional/GNN

1.2 Stack activo

El stack que esta vivo y debes usar es:

- `python -m chromahack.training.train_ppo_frontier`
- `python -m chromahack.evaluation.eval_frontier_hidden`
- `python -m chromahack.rendering.frontier_dual_renderer`
- `python -m chromahack.run_experiment -mode frontier`

1.3 Que ya no es el camino principal

Quedan como legado:

- bridge demo
- classic GhostMerc
- wrappers viejos de CNN / visual proxy

No estan borrados por completo, pero ya no forman parte de la historia principal ni del flujo recomendado.

2 Arquitectura actual

2.1 Separacion de responsabilidades

La arquitectura consolidada ya tiene esta forma:

1. **Entorno Gymnasium headless**: contiene dinamica, rewards, detector de transicion y snapshots.
2. **Entrenamiento PPO**: usa SB3 y entrena contra `proxy_reward`.
3. **Evaluacion oculta**: mide `true_reward`, gap, exploit frequency y trayectorias.
4. **Renderer PyGame**: solo consume trayectorias o una politica en inferencia.
5. **Pipeline de preferencias**: exporta pares, entrena reward model y permite reentrenar con `pref_model`.
6. **Observabilidad**: registra entropia, deriva KL y una senal PCA simple.

2.2 Archivos importantes

Archivo	Rol
`chromahack/envs/ghostmerc_frontier_env.py`	entorno principal
`chromahack/envs/territory_generator.py`	distritos, zonas y actores
`chromahack/training/train_ppo_frontier.py`	PPO canonico
`chromahack/evaluation/eval_frontier_hidden.py`	evaluacion con reward oculto
`chromahack/rendering/frontier_dual_renderer.py`	replay, master, trailer
`chromahack/gnn/feature_extractor.py`	extractor relacional para `dict` obs
`chromahack/intervention/pref_model.py`	export, train y wrapper de reward por preferencias
`chromahack/utils/metrics.py`	callback Frontier y agregados
`chromahack/metrics/frontier_observability.py`	mutual info, KL y PCA simple
`chromahack/run_experiment.py`	orquestador baseline / rlhf

3 Que artefactos concretos tienes

3.1 Demo maestra larga

La demo maestra canonica ahora esta dentro del repo:

- `artifacts/demos/frontier_master_demo_long/model/ppo_best.zip`
- `artifacts/demos/frontier_master_demo_long/eval_frontier_hidden/summary.json`
- `artifacts/demos/frontier_master_demo_long/eval_frontier_hidden/trajectories/episode_040.js`

3.2 Resultados principales de esa demo

En 50 episodios del distrito 5:

- `avg_proxy_return = 816.61`
- `avg_true_return = -306.90`
- `proxy_true_gap = 1123.51`
- `avg_armed_neutral_false_positive_rate = 0.713`
- `avg_containment_tick_count = 84.96`
- `exploit_frequency = 0.18`
- `phase_transition_rate = 0.18`
- `avg_phase_transition_step = 100`

3.3 Mejor episodio curado

El episodio maestro actual es:

- `episode_040.json`

Sus senales mas llamativas:

- `containment_tick_count = 485`
- `proxy_true_gap = 2039.68`
- `armed_neutral_false_positive_rate = 1.0`
- `phase_transition_step = 100`
- beat visual: `CONTAINMENT EXPLOIT ACTIVE`

4 Que puedes ejecutar para ver ahora mismo

4.1 Importante

El directorio recomendado es:

- `C:\Users\pc\Desktop\Proyecto_LR\PR_LR_H`

Tambien deberia funcionar desde la carpeta padre gracias al shim, pero la raiz del repo es el lugar canonico.

4.2 Para ver la demo maestra directamente

```
python -m chromahack.rendering.frontier_dual_renderer master ^
  --demo_dir artifacts/demos/frontier_master_demo_long
```

Eso abre la mejor trayectoria ya curada desde el resumen maestro.

4.3 Para ver el episodio exacto

```
python -m chromahack.rendering.frontier_dual_renderer replay ^
  --trajectory artifacts/demos/frontier_master_demo_long/
eval_frontier_hidden/trajectories/episode_040.json
```

4.4 Para verlo con presentacion mas clara

```
python -m chromahack.rendering.frontier_dual_renderer showcase ^
  --trajectory artifacts/demos/frontier_master_demo_long/
eval_frontier_hidden/trajectories/episode_040.json ^
  --slow_mo 0.8 --pause_frames 24 --alert_linger_frames 120
```

4.5 Para ver el corte editorial

```
python -m chromahack.rendering.frontier_dual_renderer trailer_cut ^
  --demo_dir artifacts/demos/frontier_master_demo_long
```

4.6 Para exportar video

```
python -m chromahack.rendering.frontier_dual_renderer trailer_cut ^
  --demo_dir artifacts/demos/frontier_master_demo_long ^
  --export_dir artifacts/captures/frontier_trailer_cut_final ^
  --video_name ghostmerc_frontier_final.mp4
```

4.7 Para ver una politica viva

```
python -m chromahack.rendering.frontier_dual_renderer policy ^
  --model_dir artifacts/demos/frontier_master_demo_long/model ^
  --model_name ppo_best ^
  --district_id 5 ^
  --camera_mode cinematic ^
  --alert_linger_frames 120 ^
  --auto_reset --max_episodes 12
```

5 Que puedes ejecutar para entrenar

5.1 Baseline MLP

```
python -m chromahack.training.train_ppo_frontier ^
  --out_dir artifacts/models/frontier_baseline ^
  --total_steps 200000 ^
  --n_envs 4 ^
  --policy_backend mlp ^
  --observation_mode flat ^
  --reward_mode proxy
```

5.2 Evaluacion del baseline

```
python -m chromahack.evaluation.eval_frontier_hidden ^
  --model_dir artifacts/models/frontier_baseline ^
  --model_name ppo_best ^
  --n_episodes 50 ^
  --save_trajectories
```

5.3 GNN / backend relacional

```
python -m chromahack.training.train_ppo_frontier ^
  --out_dir artifacts/models/frontier_gnn ^
  --total_steps 200000 ^
  --n_envs 4 ^
  --policy_backend gnn ^
  --observation_mode dict
```

5.4 Master demo largo

```
python -m chromahack.run_experiment --mode frontier --phase baseline --master_demo
```

6 Que puedes ejecutar para la ruta RLHF

6.1 Pipeline completo rapido

```
python -m chromahack.run_experiment --mode frontier --phase rlhf --quick
```

Ese comando ya fue validado en esta iteracion y hace:

1. baseline corto,
2. export de pares de preferencia,
3. entrenamiento del reward model,
4. reentrenamiento corto con `pref_model`,
5. evaluacion del run reentrenado.

6.2 Pipeline paso a paso

```
python -m chromahack.intervention.pref_model export ^
  --trajectory_dir artifacts/frontier/baseline/eval_frontier_hidden/trajectories ^
```

```
  --out_dir artifacts/frontier/preferences

python -m chromahack.intervention.pref_model train ^
  --dataset artifacts/frontier/preferences/preference_pairs.json ^
  --out_dir artifacts/frontier/preferences/model

python -m chromahack.training.train_ppo_frontier ^
  --out_dir artifacts/frontier/pref_model_run/model ^
  --reward_mode pref_model ^
  --reward_model_path artifacts/frontier/preferences/model/preference_reward_model.pt ^
  --policy_backend mlp ^
  --observation_mode flat
```

6.3 Estado actual de la ruta de preferencias

La ruta ya esta funcional, no solo en scaffolding. En la ultima validacion:

- `train_accuracy = 0.80`
- `validation_accuracy = 0.7647`

Eso confirma que el reward model sintetico ya se entrena y se puede inyectar en el entorno.

7 Que quedo validado tecnicamente

7.1 CLI

Ya funcionan:

- `python -m chromahack.training.train_ppo_frontier -help`
- `python -m chromahack.evaluation.eval_frontier_hidden -help`
- `python -m chromahack.run_experiment -help`
- `python -m chromahack.intervention.pref_model -help`
- `python -m chromahack.rendering.frontier_dual_renderer -help`
- `python -m chromahack.rendering.frontier_dual_renderer master -help`

7.2 Tests

Quedo validado:

- `python scripts/smoke_test.py`
- `python -m pytest tests/test_frontier_observation_modes.py`

Las pruebas locales cubren:

- paridad de dinamica entre `flat` y `dict`
- wrapper de reward por preferencias

7.3 Validaciones funcionales extra

Tambien quedo probado:

- entrenamiento corto con backend `gnn`
- evaluacion de un checkpoint `gnn`
- export y entrenamiento del reward model
- entrenamiento corto con `reward_mode=pref_model`
- orquestador completo `-phase rlhf -quick`

8 Como interpretar lo que ves

8.1 Proxy reward

Es lo que el agente optimiza durante entrenamiento baseline.

8.2 True reward

Es la evaluacion oculta que si representa el objetivo real.

8.3 Proxy-true gap

Si sube mucho, hay divergencia entre score visible y exito real.

8.4 Armed neutral false positive rate

Si sube, el agente esta clasificando mal a actores armados que no deberia tratar como amenaza real.

8.5 Containment tick count

Si sube, el agente esta cobrando puntos por mantener actores heridos y vigilados en vez de resolver la situacion.

8.6 Phase transition

Si aparece un `phase_transition_step`, ese es el paso donde la conducta ya entro a un regimen diferente y mas explotativo.

9 Resumen rapido para explicarlo facil

9.1 En una frase

Entrene un agente tactico con una recompensa proxy corrupta, y el proyecto muestra como puede subir mucho su score mientras fracasa en el objetivo real.

9.2 En 30 segundos

GhostMerc Frontier Territory es un testbed de RL para estudiar reward hacking. El agente vive en un distrito con civiles, hostiles y actores ambiguos. PPO lo entrena con una senal proxy que premia headshots, threat tags y containment ticks. El objetivo real oculto es proteger civiles, estabilizar el territorio y evitar falsos positivos. En la demo larga, el agente logra proxy muy alto y true reward muy negativo, lo que demuestra desalineacion visible y medible.

9.3 Tres numeros que conviene memorizar

- `816.61`: proxy medio de la demo larga
- `-306.90`: true reward medio de la demo larga
- `2039.68`: gap del mejor episodio

10 Que hacer despues

Si lo que quieres es **ver** el proyecto, ejecuta:

- `master`
- `showcase`
- `trailer_cut`

Si lo que quieres es **investigarlo**, ejecuta:

- baseline MLP
- backend GNN
- fase RLHF sintetica

Si lo que quieres es **explicarlo facil**, muestra:

- el replay del `episode_040`
- el gap proxy–true
- la idea de que el agente parece competente pero optimiza el objetivo equivocado

11 Cierre

En este momento tu proyecto ya no es solo una idea o una demo visual. Ya tienes:

- entorno Frontier consolidado
- baseline reproducible
- renderer utilizable
- demo maestra larga dentro del repo
- backend relacional habilitado
- ruta inicial RLHF funcionando
- documentacion y paper separados

Lo mas importante: ya puedes **mostrar**, **explicar** y **seguir investigando** sobre el mismo stack, sin rehacer el proyecto desde cero.